1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that enhances Large Language Models (LLMs) by providing specific external knowledge at runtime. Instead of relying only on training data, RAG **retrieves** relevant documents from a private knowledge base to help generate accurate responses.

Why RAG is Needed

- **Accuracy**
  Prevents hallucinations by grounding answers in factual documentation.
- **Up-to-Date Information**
  Lets the AI support new products or manuals without expensive retraining.
- **Context Awareness**
  Supplies technical details unique to a brand, such as reset procedures or port configurations.

Use Case Overview

The **TechNova Support Bot** is a specialized assistant for the **X1000 Router**. It serves as a first line of support by:

- Handling common technical troubleshooting queries
- Retrieving answers from documentation
- Escalating sensitive or complex issues to human agents

---

2. System Architecture

Overall Architecture

The system follows a **Modular RAG Pipeline** integrated with **LangGraph** orchestration.

Layers

1. Ingestion Layer

Processes PDF manuals into searchable vector embeddings.

2. Storage Layer

Uses **ChromaDB** for persistent local vector storage.

3. Agentic Layer

Graph-based workflow for:

- Intent classification
- Retrieval
- Relevance grading
- Response generation

4. Interface Layer

Streamlit web application for end-user interaction.

Hybrid RAG + Graph-Based Agent

Unlike standard RAG:

Question → Retrieve → Answer

this project uses an **Agentic RAG** approach where the graph can:

- Branch
- Loop
- Self-correct
- Route to humans when needed

It acts as its own quality control system.

---

3. Design Decisions

Chunking Strategy

- **Chunk Size:** 800 characters
- **Overlap:** 200 characters

Rationale

- Large enough to contain full troubleshooting procedures
- Overlap prevents technical details from splitting across chunk boundaries

---

Embedding Model Choice

Model

all-MiniLM-L6-v2

Rationale

- Efficient for local deployment
- Strong performance on technical documentation
- Low computational overhead

Retrieval + Grading

Approach

- Similarity Search
- AI-based relevance grading

Rationale

Vector search alone may return noisy results.

Adding a grading node ensures the generator sees only truly relevant context.

---

4. Workflow Explanation (LangGraph)

Why LangGraph?

LangGraph enables **stateful, multi-step AI workflows** with conditional logic.

Supports:

- Human-in-the-Loop (HITL)
- Corrective RAG
- Stateful routing
- Dynamic decision-making

Nodes and Responsibilities

`classify`

- Detects whether query is:
    - Technical
    - Account-related

`retrieve`

- Fetches top **3** chunks from ChromaDB.

`grade`

- Evaluates whether retrieved chunks are relevant.

`generate`

- Produces final grounded answer using the LLM.

`human_escalate`

- Prepares workflow state for human intervention.

---

5. Conditional Logic

The system uses **Decision Edges** to route conversations.

Routing Rules

Intent Detection

If Intent = Account
→ Escalate

Relevance Grading

If Grade = No
→ Escalate

Successful Answer Path

If Intent = Technical
AND Grade = Yes
→ Generate Answer

---

6. Human-in-the-Loop (HITL)

Escalation Triggers

Escalate when:

- Ambiguous or non-technical queries

- Refund, legal, or complaint-related requests
- Manual lacks the answer
- Low confidence grading results

Process Flow

1. AI flags:

escalate = True

2. UI shows **Human Agent Panel**
3. Human agent enters manual response
4. Response is injected into chat history and UI refreshes

---

7. Challenges & Trade-Offs

Accuracy vs Speed

- Adding a grading node introduces a few seconds of latency
- But greatly reduces incorrect technical advice

Hallucination Control

The model is instructed to use **only provided context**.

Trade-off:

- Less flexible for general questions
- Much safer for technical support

---

8. Testing Strategy

Sample Test Queries

1. Technical Query

Input:
"How do I reset my router?"

Expected:
Step-by-step reset guide

---

2. Specific Product Query

Input:
"What is the default SSID?"

Expected:

TechNova_X1000

---

3. Account Query

Input:
"I want to cancel my warranty."

Expected:
Route to human escalation

---

4. Out-of-Scope Query

Input:
"How do I make a pizza?"

Expected:
Escalate to human *(outside domain)*

---

9. Future Enhancements

Planned Improvements

Multi-Document Support

Support manuals for multiple router models.

User Feedback Loop

Add:

- 👍 Thumbs up
- 👎 Thumbs down

to improve retrieval quality.

Monitoring

Integrate **LangSmith** for:

- Graph tracing
- Decision monitoring
- Debugging

Deployment

Containerize with **Docker** for deployment on:

- AWS
- GCP

Summary

This project combines:

- **RAG for grounded retrieval**
- **LangGraph for agentic workflows**
- **HITL for safety escalation**

- **Streamlit for interactive support delivery**